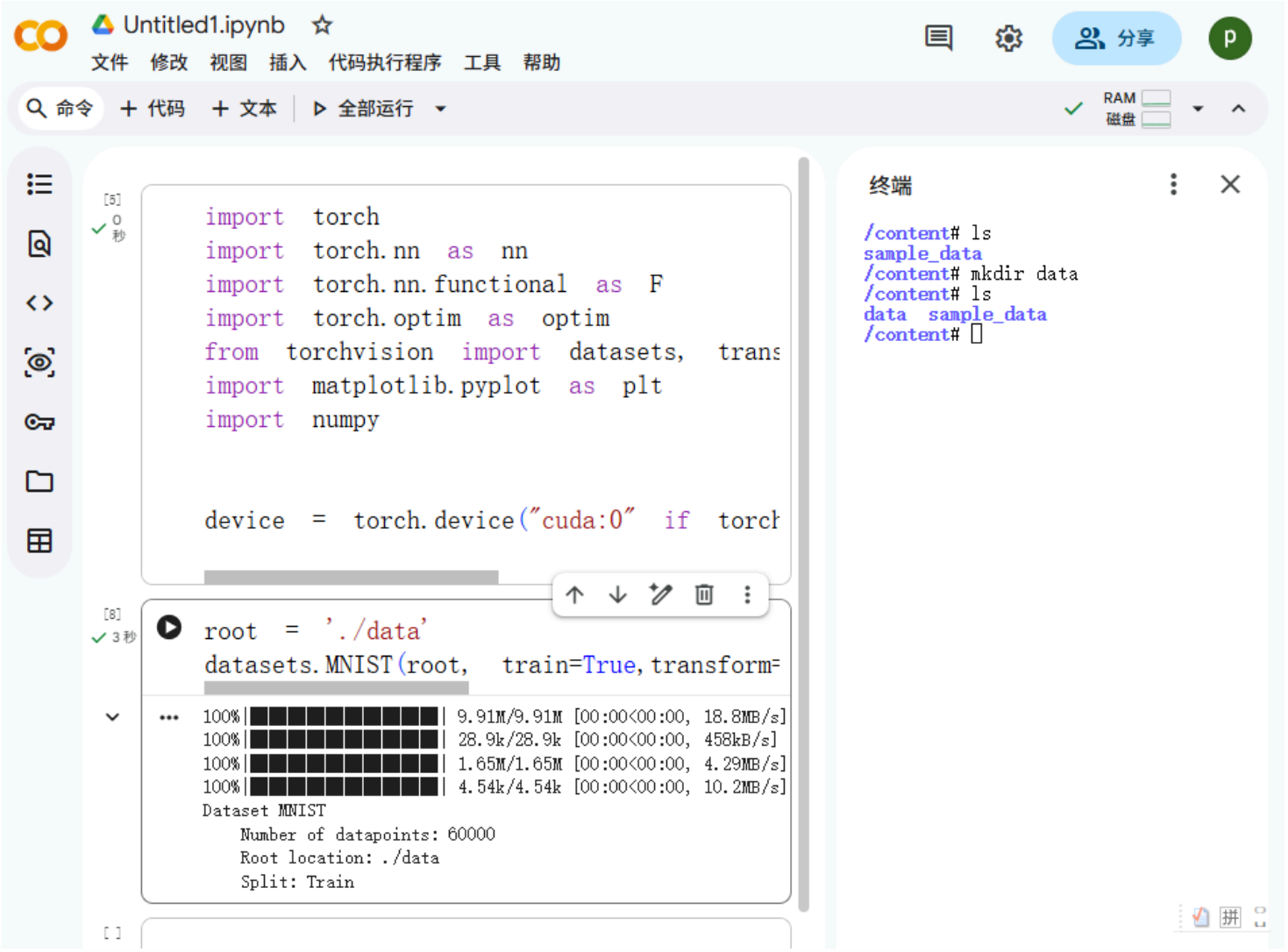
姓名和学号?	**程鹏炜，24130061003**
本实验属于哪门课程?	中国海洋大学25秋《软件工程原理与实践》
实验名称?	实验2：深度学习基础
博客链接:	https://blog.csdn.net/cpw123_/article/details/168344448?sharetype=blogdetail&sharerId=168344448&sharerefer=PC&sharesource=cpw123_&spm=1011.2480.3001.8118

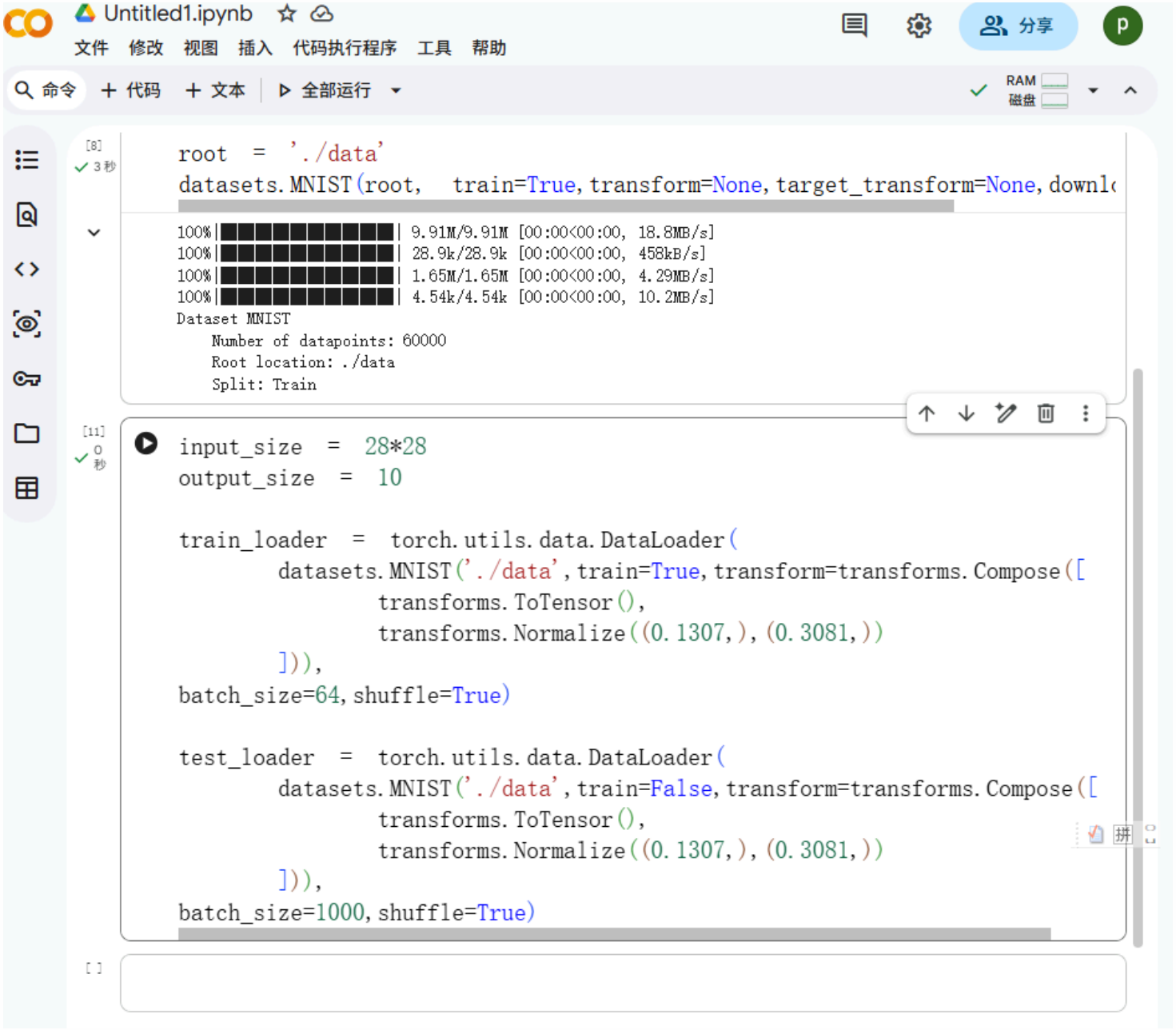
一、实验内容

代码练习和思考

1.手写数字识别任务-数据处理：

加载下载数据集，train=true下载训练集然后对数据做transform操作基本上就是改变一些维度让或者裁剪预处理等等，这里就是简单的处理让他变成tensor按照对应维度的归一化然后还有规定batch数量。


Untitled1.ipynb
文件 修改 视图 插入 代码执行程序 工具 帮助
分享
命令 + 代码 + 文本 全部运行
RAM
磁盘
[5]
0 秒
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, trans
import matplotlib.pyplot as plt
import numpy
device = torch.device("cuda:0" if torch
[8]
3 秒
root = './data'
datasets.MNIST(root, train=True,transform=
100%| 9.91M/9.91M [00:00<00:00, 18.8MB/s]
100%| 28.9k/28.9k [00:00<00:00, 458kB/s]
100%| 1.65M/1.65M [00:00<00:00, 4.29MB/s]
100%| 4.54k/4.54k [00:00<00:00, 10.2MB/s]
Dataset MNIST
Number of datapoints: 60000
Root location: ./data
Split: Train
[ ]
终端
/content# ls
sample_data
/content# mkdir data
/content# ls
data sample_data
/content#

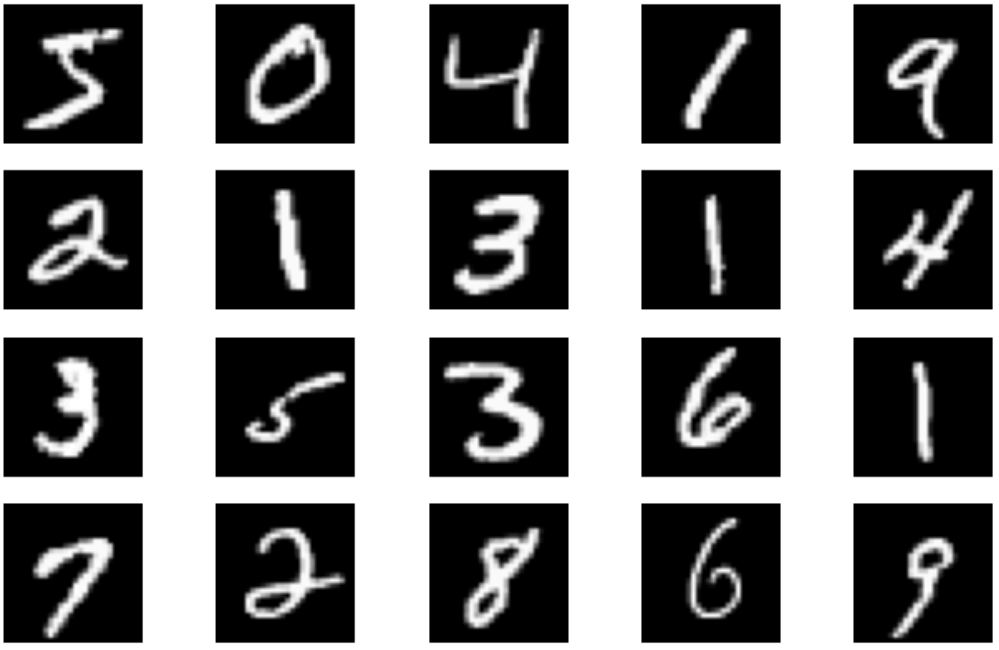


划分训练和测试集，基本上都是同样的操作注意shuffle与否，基本上都需要去打乱数据顺序防止出现学习到数据排列规律。

```
    ])),
batch_size=1000, shuffle=True)
```

[13] ✓ 1秒

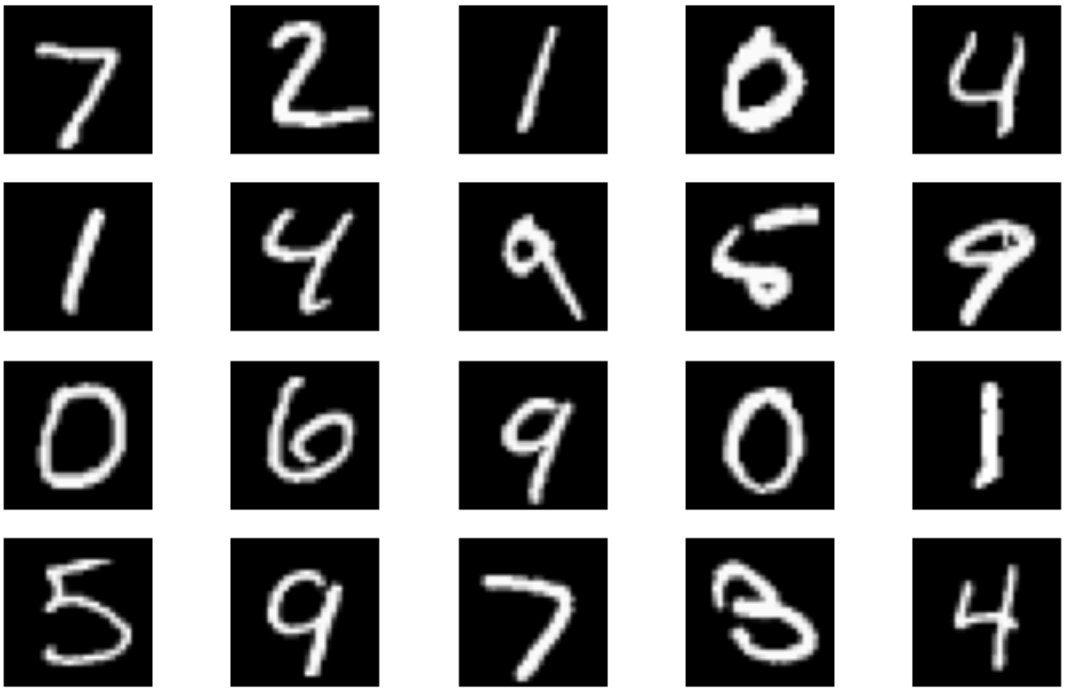
```
plt.figure(figsize=(8,  5))
for  i  in  range(20):
    plt.subplot(4,5,i+1)
    image,  _  =  train_loader.dataset.__getitem__(i)
    plt.imshow(image.squeeze().numpy(),'gray')
    plt.axis('off');
```



[ ]

展示训练集，然后再去看测试集能发现同样的数据是长得不一样的。比如这个7

```
plt.figure(figsize=(8,  5))
for  i  in  range(20):
    plt.subplot(4,5,i+1)
    image,  _  =  test_loader.dataset.__getitem__(i)
    plt.imshow(image.squeeze().numpy(),'gray')
    plt.axis('off');
```



为什么使用logsoftmax可能更适配吧不清楚为什么不直接使用softmax

2.模型构建：

```
class FC2Layer(nn.Module):
    def __init__(self, input_size, n_hidden, output_size):
        super(FC2Layer,self).__init__()
        self.input_size = input_size
        self.network = nn.Sequential(
                nn.Linear(input_size,n_hidden),
                nn.ReLU(),
                nn.Linear(n_hidden,n_hidden),
                nn.ReLU(),
                nn.Linear(n_hidden,output_size),
                nn.LogSoftmax(dim=1)
        )
    def forward(self, x):
        x = x.view(-1,self.input_size)
        print(x.cpu().numpy().shape)
        return self.network(x)

    class CNN(nn.Module):
        def __init__(self, input_size, n_feature, output_size):
            super(CNN, self).__init__()
            self.n_feature = n_feature
            self.conv1 = nn.Conv2d(in_channels=1, out_channels=n_feature
            self.conv2 = nn.Conv2d(n_feature,n_feature,kernel_size=5)
            self.fc1 = nn.Linear(n_feature*4*4, 50)#这里×4*4是通过kerne
            self.fc2 = nn.Linear(50, 10)

        def forward(self, x, verbose=False):
            x = self.conv1(x)
            x = F.relu(x)
            x = F.max_pool2d(x,kernel_size=2)
            x = self.conv2(x)
                relu(x)
                max_pool2d(x,kernel_size=2)
```

可以看到在普通的神经网络上和之前没什么区别，主要是cnn里面我们发现每一层传输的过程中特征通道维数是变化的而且是根据卷积计算的过程中kernel滤波器去做卷积运算和padding带来的变化，以及池化的时候做max还是avg都带来对应维度的变化。

整理了一点：

卷积层（无填充 padding=0，步长 stride=1）：
输出边长 = 输入边长 - kernel_size + 1
如果有填充和步长，通用公式是：
输出边长 = (输入边长 + 2×padding - kernel_size) / stride + 1，取整数部分

最大池化层：
输出边长 = 输入边长 / kernel_size（一般 stride 等于 kernel_size）

3.训练测试：

```
[18]
0
秒
def train(model):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optim.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optim.step()
        if batch_idx % 100 == 0:
            print('Train:[{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                    batch_idx * len(data), len(train_loader.dataset),
                    100. * batch_idx / len(train_loader), loss.item()
            ))

def test(model):
    model.eval()
    test_loss = 0
    correct = 0
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        output = model(data)
        test_loss += F.nll_loss(output, target, reduction='sum').item()
        pred = output.data.max(1,keepdim=True)[1]
        correct += pred.eq(target.data.view_as(pred)).cpu().sum().item()
    test_loss /= len(test_loader.dataset)
    accuracy = 100. * correct/len(test_loader.dataset)
    print('\nTest set: Average Loss: {:.4f},Accuracy:{}/{}({:.0f}%)\n'.f
            test_loss, correct, len(test_loader.dataset),
            accuracy
```

终端 ✓ 19:46 Python 3

值得注意的是grad每次归零然后就是前向传播反向传播即可。更新优化器更新参数打印日志。

发现报错了，忘记写get parameter函数了：

```
import  torch
import  torch.nn  as  nn
import  torch.nn.functional  as  F
import  torch.optim  as  optim
from  torchvision  import  datasets,  transforms
import  matplotlib.pyplot  as  plt
import  numpy

def  get_n_params(model):
    np  =  0
    for  p  in  list(model.parameters()):
        np  +=  p.nelement()
    return  np

device  =  torch.device("cuda:0"  if  torch.cuda.is_available()else  "cpu"
```

```
root  =  './data'
datasets.MNIST(root,  train=True,transform=None,target_transform=None,down
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 18.8MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 458kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.29MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 10.2MB/s]
Dataset MNIST
    Number of datapoints: 60000
    Root location: ./data
    Split: Train
```

```
n_hidden = 8
model_fnn = FC2Layer(input_size, n_hidden, output_size)
model_fnn.to(device)
optimizer = optim.SGD(model_fnn.parameters(),lr=0.01,momentum=0.5)
print('Number of parameters: {}'.format(get_n_params(model_fnn)))

train(model_fnn)
test(model_fnn)
```

```
Number of parameters: 6442
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_3236/3691181146.py in <cell line: 0>()
      5 print('Number of parameters: {}'.format(get_n_params(model_fnn)))
      6
----> 7 train(model_fnn)
      8 test(model_fnn)

/tmp/ipykernel_3236/3345059642.py in train(model)
      3   for batch_idx, (data,target) in enumerate(train_loader):
      4     data, target = data.to(device),target.to(device)
----> 5     optim.zero_grad()
      6     output = model(data)
      7     loss = F.nll_loss(output,target)

AttributeError: module 'torch.optim' has no attribute 'zero_grad'
```

结果准确率85左右发现不对劲啊cnn应该没这么低然后就发现test用的model是fnn的不是cnn的

```
(64, 784)
(64, 784)
(64, 784)
(64, 784)
(64, 784)
(64, 784)
(64, 784)
(64, 784)
(64, 784)
(64, 784)
(32, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)

Test set: Average Loss: 0.5103,Accuracy:8508/10000(85%)
```

```
n_features = 6
model_cnn = CNN(input_size, n_features, output_size)
model_cnn.to(device)
optimizer = optim.SGD(model_cnn.parameters(), lr=0.01, momentum=0.5)
print('Number of parameters: {}'.format(get_n_params(model_cnn)))

train(model_cnn)
test(model_fnn)
```

```
Number of parameters: 6422
Train:[0/60000 (0%)]    Loss: 2.301337
Train:[6400/60000 (11%)]        Loss: 2.212125
Train:[12800/60000 (21%)]       Loss: 0.738858
Train:[19200/60000 (32%)]       Loss: 0.600706
Train:[25600/60000 (43%)]       Loss: 0.407764
Train:[32000/60000 (53%)]       Loss: 0.286551
Train:[38400/60000 (64%)]       Loss: 0.316317
Train:[44800/60000 (75%)]       Loss: 0.202716
Train:[51200/60000 (85%)]       Loss: 0.104152
Train:[57600/60000 (96%)]       Loss: 0.173967
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)
(1000, 784)

Test set: Average Loss: 0.5103,Accuracy:8508/10000(85%)
```

```
n_features = 6
model_cnn = CNN(input_size, n_features, output_size)
model_cnn.to(device)
optimizer = optim.SGD(model_cnn.parameters(), lr=0.01, momentum=0.5)
print('Number of parameters: {}'.format(get_n_params(model_cnn)))

train(model_cnn)
test(model_cnn)
```

```
Number of parameters: 6422
Train:[0/60000 (0%)]    Loss: 2.304808
Train:[6400/60000 (11%)]        Loss: 1.123704
Train:[12800/60000 (21%)]       Loss: 0.531426
Train:[19200/60000 (32%)]       Loss: 0.304839
Train:[25600/60000 (43%)]       Loss: 0.349864
Train:[32000/60000 (53%)]       Loss: 0.575223
Train:[38400/60000 (64%)]       Loss: 0.340000
Train:[44800/60000 (75%)]       Loss: 0.241882
Train:[51200/60000 (85%)]       Loss: 0.276974
Train:[57600/60000 (96%)]       Loss: 0.203977

Test set: Average Loss: 0.1610,Accuracy:9507/10000(95%)
```

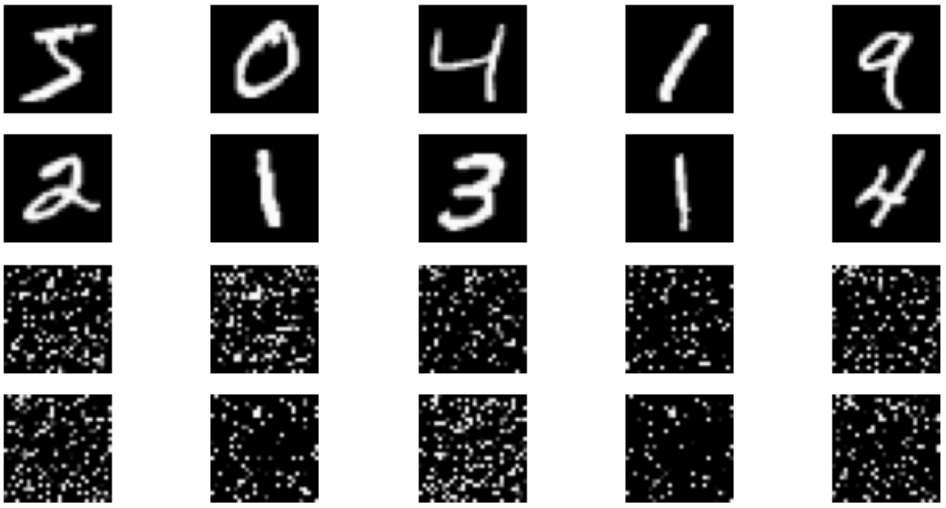
修改之后就对了95%

打乱之后的照片样子---像素之间乱窜最后准确率cnn大幅度下降了

```
Test set: Average Loss: 0.1610,Accuracy:9507/10000(95%)
```

[36] ✓ 1秒

```
perm = torch.randperm(784)
plt.figure(figsize=(8, 4))
for i in range(10):
    image,_ = train_loader.dataset.__getitem__(i)
    image_perm = image.view(-1, 28*28).clone()
    image_perm = image_perm[:,perm]
    image_perm = image_perm.view(-1,1,28,28)
    plt.subplot(4,5,i+1)
    plt.imshow(image.squeeze().numpy(),'gray')
    plt.axis('off')
    plt.subplot(4,5,i+11)
    plt.imshow(image_perm.squeeze().numpy(),'gray')
    plt.axis('off')
```



[ ]

```
perm = torch.randperm(784)
n_features = 6
model_cnn = CNN(input_size,n_features,output_size)
model_cnn.to(device)
optimizer = optim.SGD(model_cnn.parameters(),lr=0.01,momentum=0.5)
print('Number of parameterms:{}'.format(get_n_params(model_fnn)))

train_perm(model_cnn, perm)
test_perm(model_cnn, perm)
```

```
Number of parameterms:6442
Train:[0/60000 (0%)]    Loss: 2.352890
Train:[6400/60000 (11%)]        Loss: 2.233334
Train:[12800/60000 (21%)]       Loss: 2.070761
Train:[19200/60000 (32%)]       Loss: 1.588356
Train:[25600/60000 (43%)]       Loss: 1.270905
Train:[32000/60000 (53%)]       Loss: 0.999620
Train:[38400/60000 (64%)]       Loss: 0.744284
Train:[44800/60000 (75%)]       Loss: 0.749543
Train:[51200/60000 (85%)]       Loss: 0.702559
Train:[57600/60000 (96%)]       Loss: 0.569655

Test set: Average Loss: 0.5641,Accuracy:8205/10000(82%)
```

4.cifar10数据集的实验-数据准备与处理：

```
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))]
)
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64,
                                          shuffle=True, num_workers=2)
testset = torchvision.datasets.CIRFAR(root='./data', train=False,
                                      download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, ba
                                         shuffle=False, num_workers=2)
classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

```
79%|████████  | 135M/170M [38:04<09:39, 62.0kB/s]
```

Loading...

```
class Net(nn.Module):
    def __init__(self)
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

net = Net().to(device)
```

失败。此文件是远程更新的或在其他标签页中更新的。 显示差异

终端 11:42 正在恢复执行

数据集下载非常慢而且临时存储在临时磁盘上面下次连接的时候就没了数据集。连接谷歌云硬盘本地下载传上去然后每次只要解压到notebook的对应位置即可。

5.model

```
class  Net(nn.Module):
    def  __init__(self):
        super(Net,  self).__init__()
        self.conv1  =  nn.Conv2d(3, 6, 5)
        self.pool  =  nn.MaxPool2d(2, 2)
        self.conv2  =  nn.Conv2d(6, 16, 5)
        self.fc1  =  nn.Linear(16*5*5, 120)
        self.fc2  =  nn.Linear(120, 84)
        self.fc3  =  nn.Linear(84, 10)
    def  forward(self, x):
        x  =  self.pool(F.relu(self.conv1(x)))
        x  =  self.pool(F.relu(self.conv2(x)))
        x  =  torch.flatten(x,  1)
        x  =  F.relu(self.fc1(x))
        x  =  F.relu(self.fc2(x))
        x  =  self.fc3(x)
        return  x

net  =  Net().to(device)
criterion  =  nn.CrossEntropyLoss()
optimizer  =  optim.Adam(net.parameters(), lr=0.01)
```

这里要加一个flatten才行也可以像实验里面用view做变换但是感觉flatten不用脑子哈哈。

```
images, labels = iter(testloader).next()
imshow(torchvision.utils.make_grid(images))
for j in range(8):
    print(classes[labels[j]])
```

```
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_924/2558545131.py in <cell line: 0>()
----> 1 images, labels = iter(testloader).next()
      2 imshow(torchvision.utils.make_grid(images))
      3 for j in range(8):
      4   print(classes[labels[j]])

AttributeError: '_MultiProcessingDataLoaderIter' object has no attribute 'next'
```

这里不知道是不是版本问题应该改成直接调用next而不是调用iter的next方法。

6.mathematic

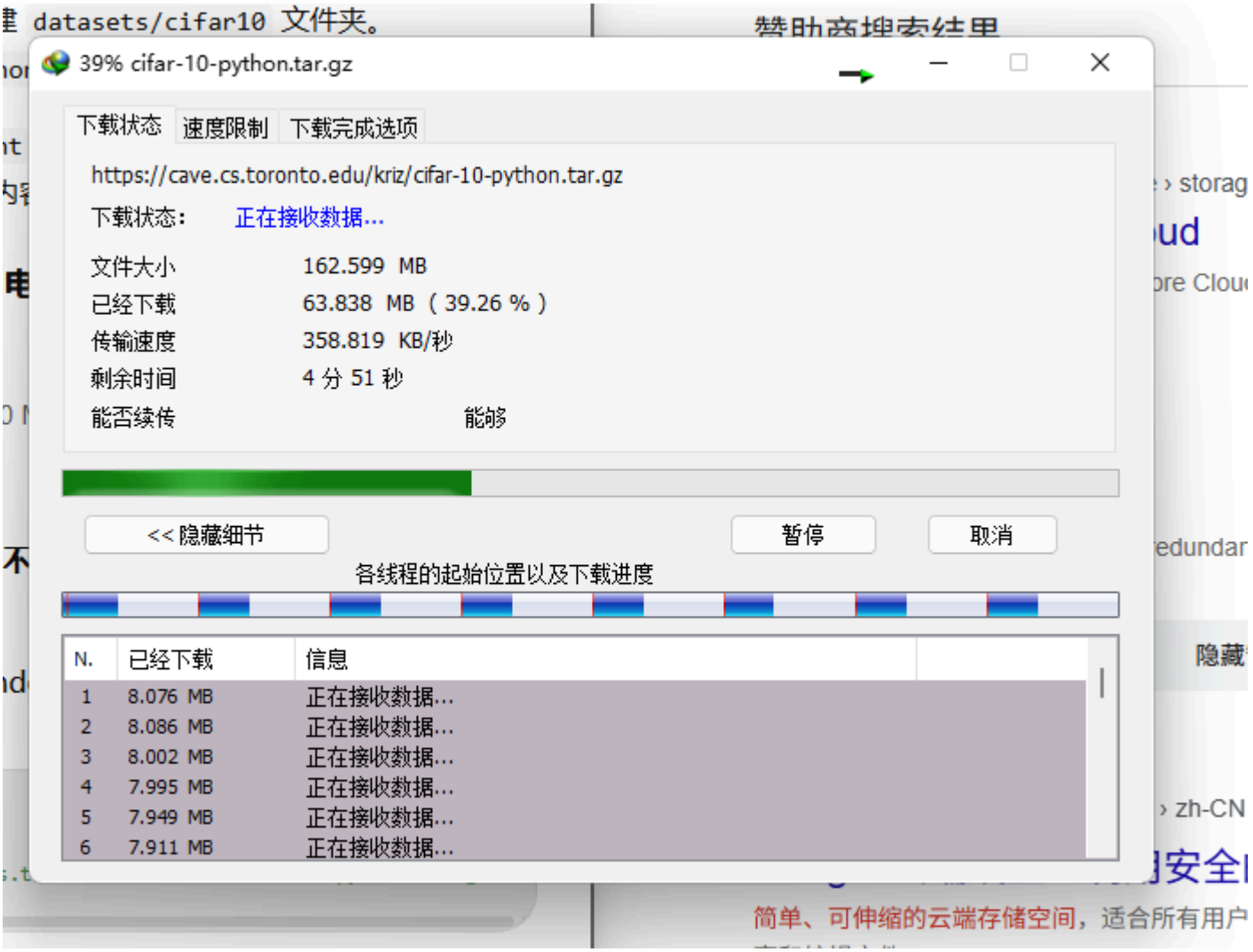
先看看数据长什么样再训练按理来说这个步骤应该放在data部分的但是不改了，**大家也可以按照data-model-mathematic-experiment-conclusion的架构组织文章**尤其是这种深度学习类型的就这几个重点。

训练之后发现准确率只有41%非常奇怪找不到问题

```
correct = 0
total = 0
for data in testloader:
    images, labels = data
    images, labels = images.to(device),labels.to(device)
    outputs = net(images)
    _, predicted = torch.max(outputs.data,1)
    total += labels.size(0)
    correct += (predicted == labels).sum().item()
    print('Accuracy of the network on the 10000 test images: %d %%
```

```
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
Accuracy of the network on the 10000 test images: 41 %
```

然后打印发现数据集少了几个batch估计下载的时候出了问题，所以我后面用本地加云盘方式重新传了一边数据：

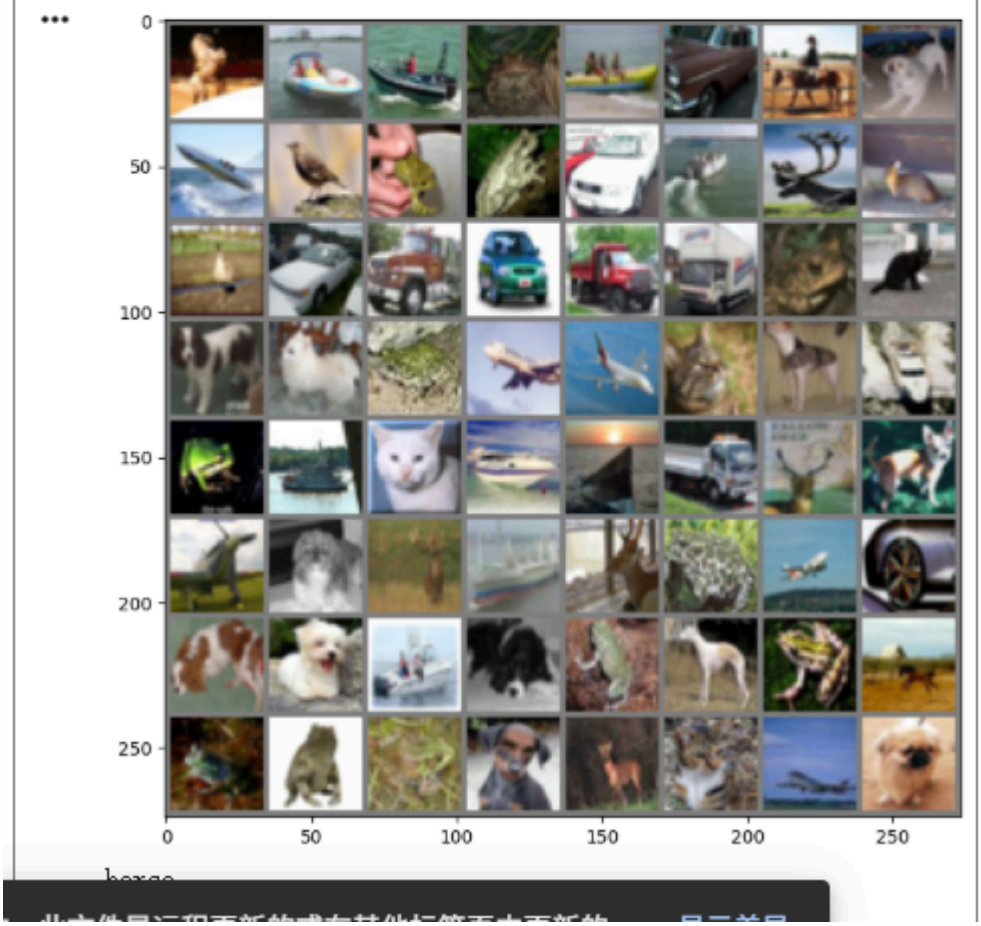



```
/content/drive/MyDrive/

/content/drive/MyDrive/datasets/cifar10/
```

```
total 12K
drwx------ 2 root root 4.0K Sep 30 11:23 'Colab Notebooks'
drwx------ 3 root root 4.0K Oct 15 09:27  datasets
drwx------ 2 root root 4.0K Sep 30 11:26  se-deep-learning
total 163M
-rw------- 1 root root 163M Oct 15 09:37 cifar-10-python.tar.g
```

```
    plt.figure(figsize=(8,8))
    img = img/2 + 0.5
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1,2,0)))
    plt.show()
ges, labels = next(iter(trainloader))
now(torchvision.utils.make_grid(images))
  j in range(8):
print(classes[labels[j]])
```



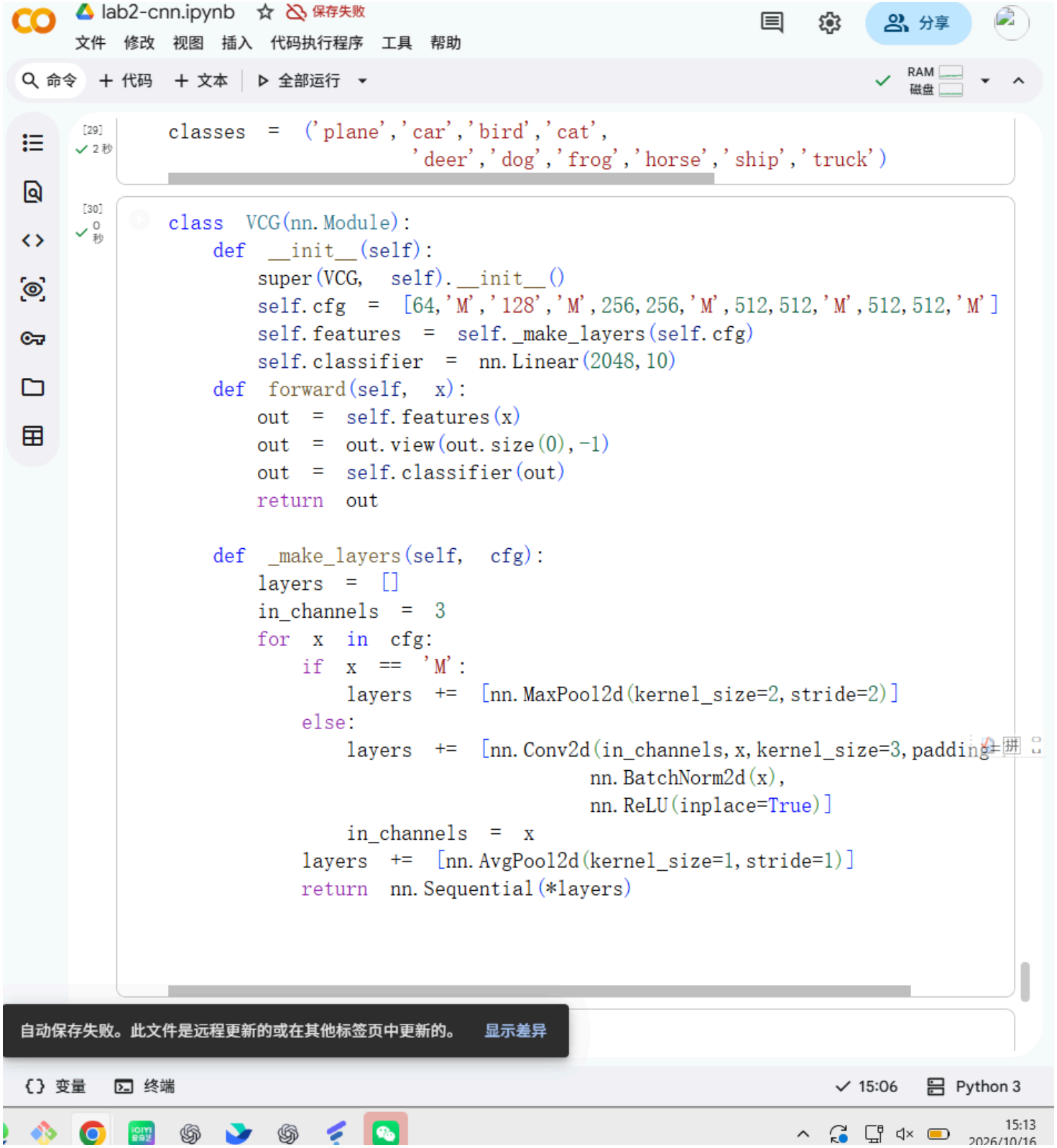
结果发现准确率还是只有50%。最后检查一圈发现是lr学习率少打一个0，设的太大了

```
conv2   =   nn.Conv2d(6, 16, 5
fc1   =   nn.Linear(16*5*5, 120)
fc2   =   nn.Linear(120, 84)
fc3   =   nn.Linear(84, 10)
ard(self, x):
 self.pool(F.relu(self.conv1(x)))
 self.pool(F.relu(self.conv2(x)))
 torch.flatten(x,   1)
 F.relu(self.fc1(x))
 F.relu(self.fc2(x))
 self.fc3(x)
n   x

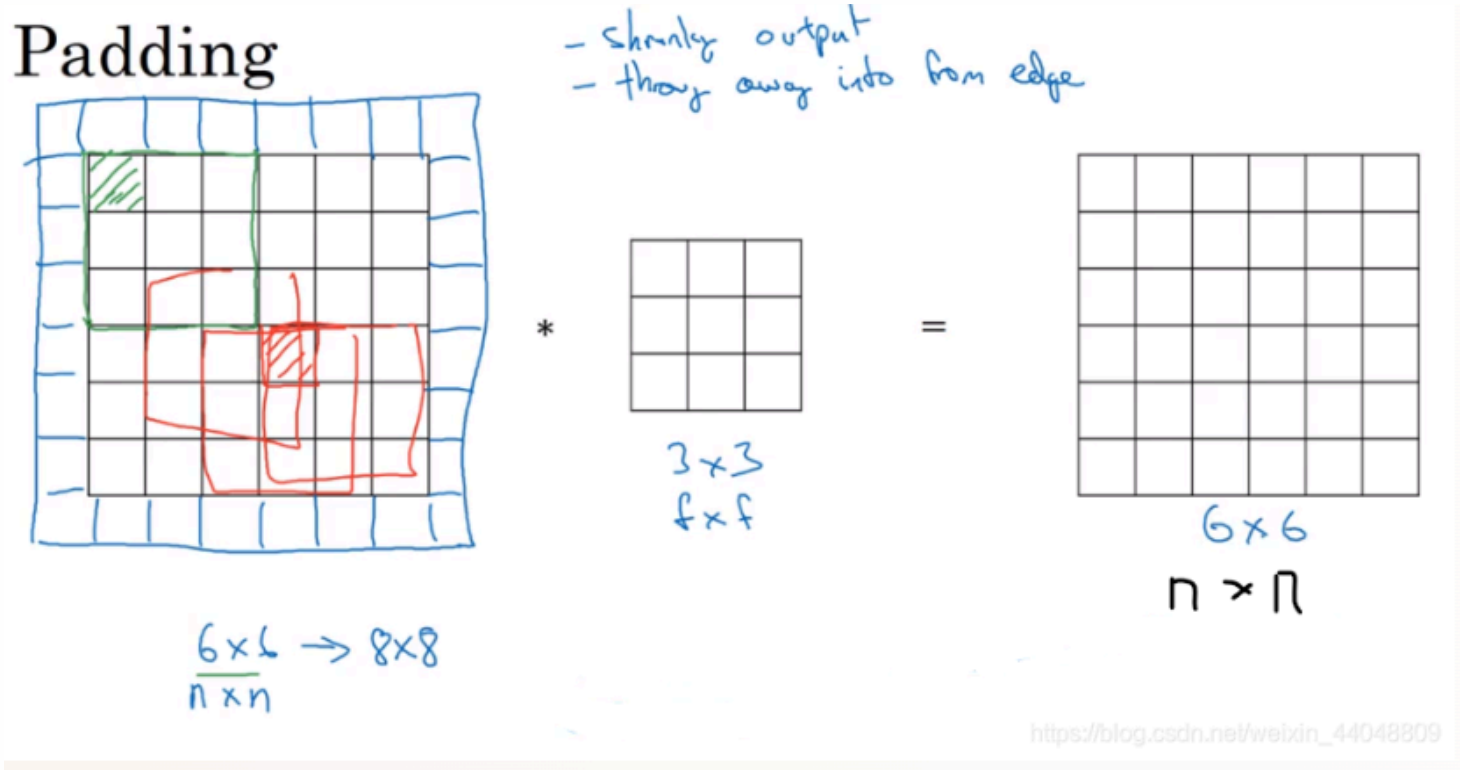
.to(device)
 nn.CrossEntropyLoss()
 optim.Adam(net.parameters(), lr=0.001)
```

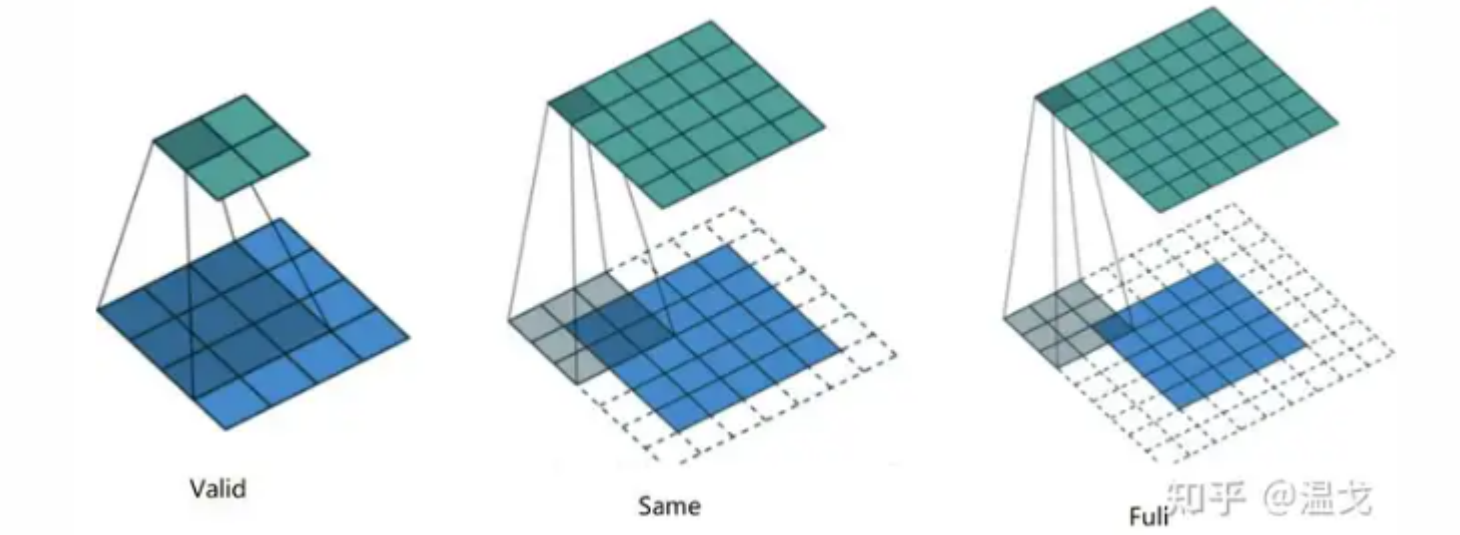
7.VCG-model：

前面数据集都一样我们直接看一下model部分有什么不同：



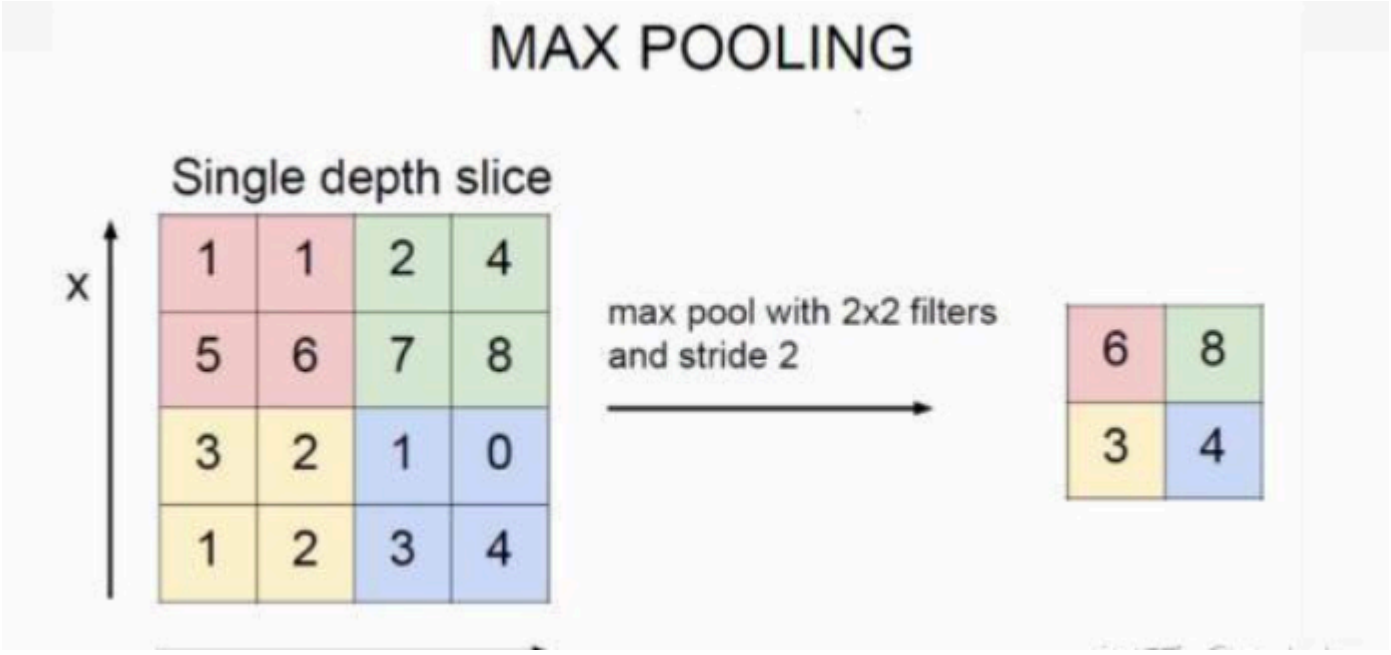
首先多出来了padding 填充，做卷积的时候当kernel大小取得不合适或者必须这么取的时候一定会出现一个kernel里面并不是我所需要的一个矩阵而是比如kernel 3*3 然后实际的数据大小不够刚刚好做几个卷积就需要我们填充



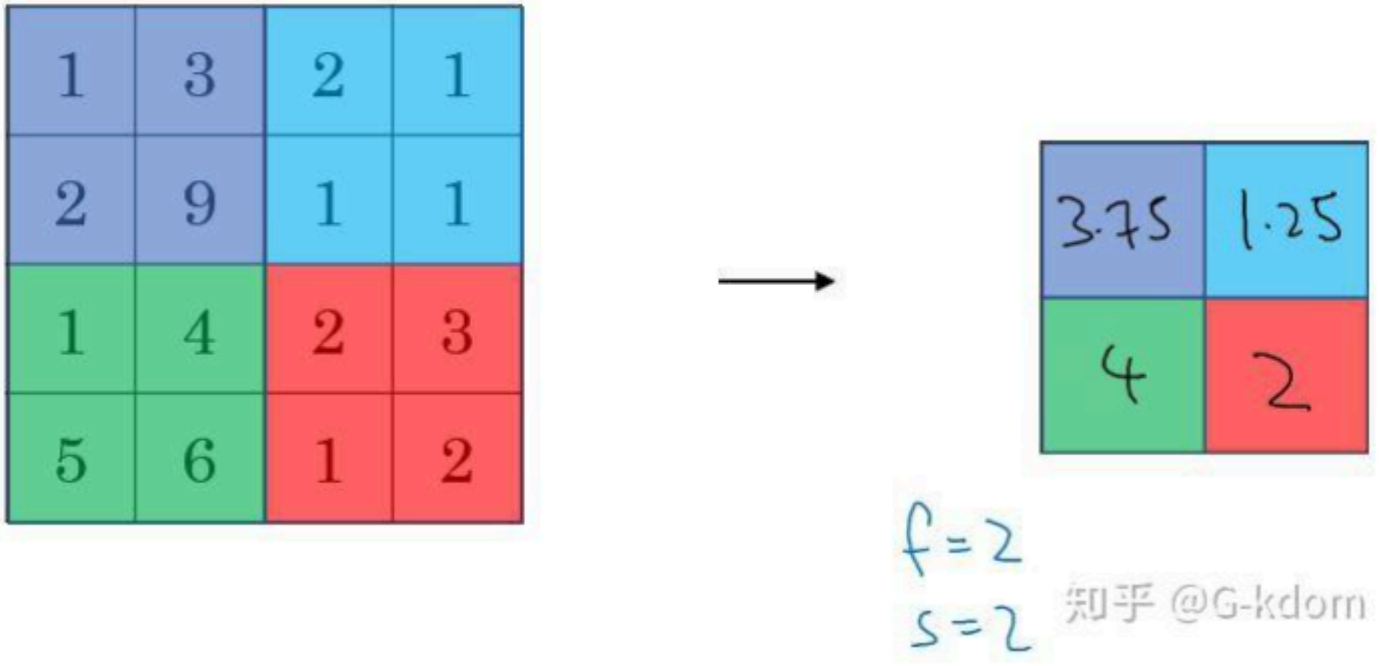


简单看一下几种类型。

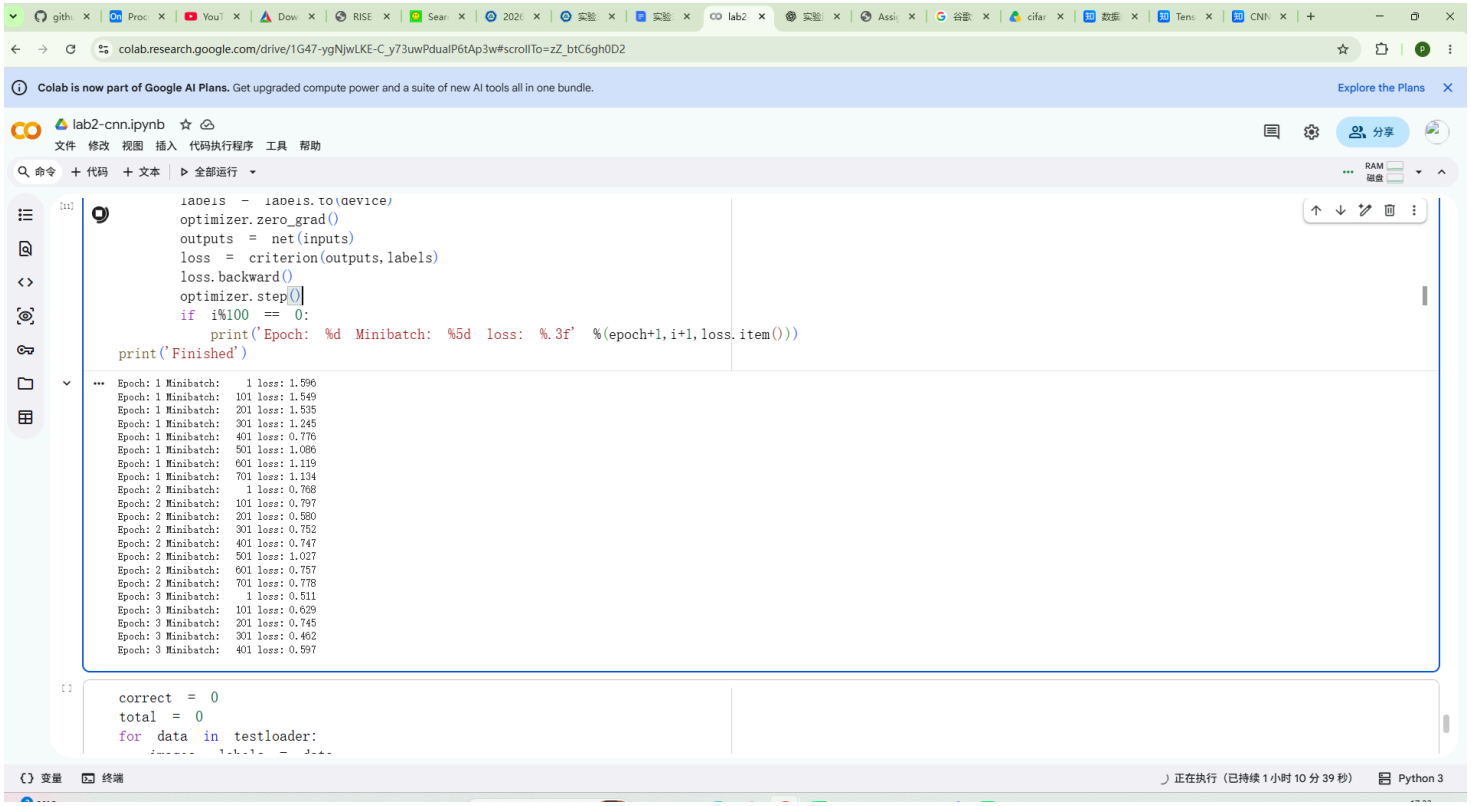
然后除了padding就是池化了，之前都是无脑maxpooling在输出的里面用一个kernel去去这里面最大值作为结果的位置中的值



然后vcg里面加入了avgpooling不再是简单的取最大值虽然最大值一定程度上能反应这一区域的特征，而实际上cnn就是一个关注局部特征的方法，但是很多时候关注局部特征会出现盲人摸象的情况，所以一直都在追求泛化上面越走越远。

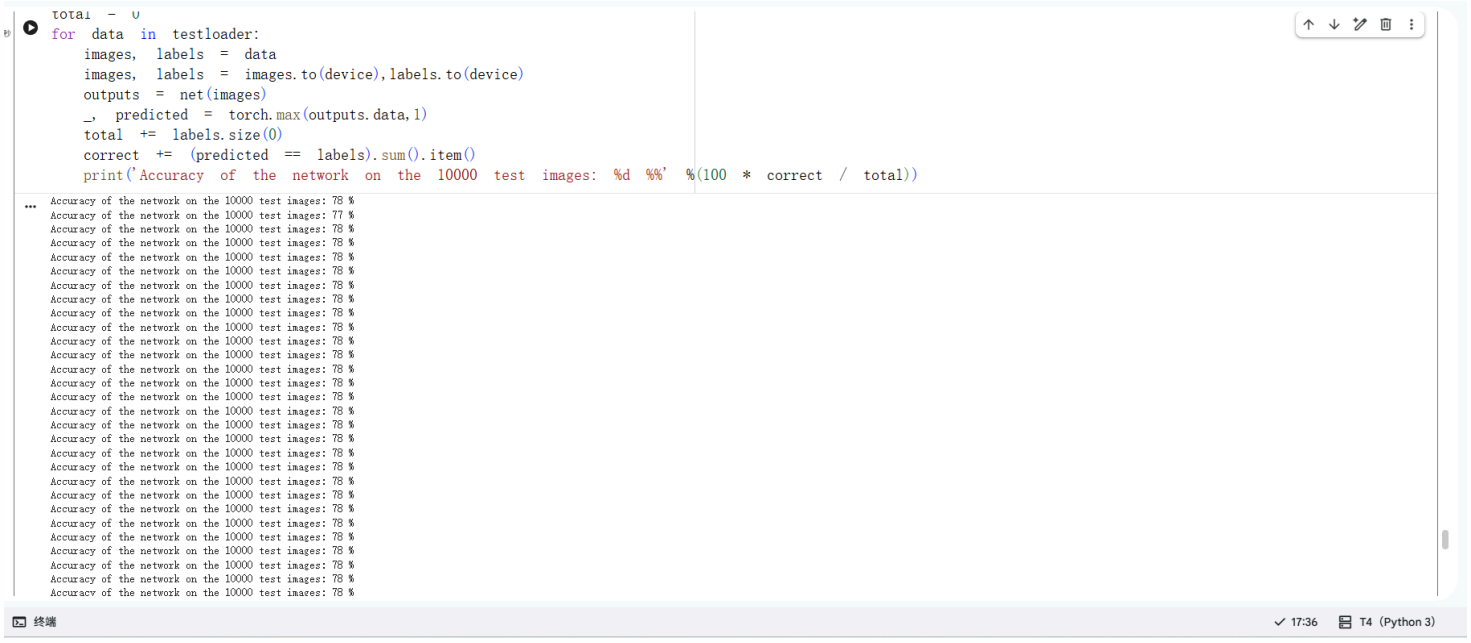


总结来说池化就是采样，采样好理解吧，取出样本来观察！哈哈。



最后这个为什么训练这么慢是什么情况是不是gpu没用啊?

PyTorch版本: 2.11.0+cpu CUDA是否可用: False PyTorch CUDA版本: None 当前设备变量: cpu 当前PyTorch没有检测到可用的CUDA GPU
我是傻逼没开gpu。点击更改运行时换成gpu。



只有七十八哈哈总是差点。

二、问题总结与体会

I.dataloader 里面shuffle取不同的值的区别

shuffle是打乱图片顺序的意思，就是洗牌，如果不shuffle，原始数据集的图片可能是按照顺序放的比如dog一百张然后cat100张，那如果划分batch的时候这一类完全是dog或者完全是cat，很有可能模型就学到了这汇总顺序然后导致泛化很差，训练集上表现的非常高的准确率到测试集上就拉跨了。。。

II.transform里，取了不同的值有什么区别

```
    datasets.MNIST('./data',train=True,transform=transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,),(0.3081,))
    ])),
```

那这个举例子：transform里面对训练数据的预处理，首先转化成张量，可以理解为向量或者矩阵，然后需要归一化，比如像素值rgb是0-255要归一化到0-1.然后是标准化，这个就是和正态分布有关系了

$$x_{\text{new}} = \frac{x - \mu}{\sigma}$$

这样归一化要知道像素值的平均值和标准差才能这样标准化。这些值是发布数据集的嘉豪给出来的

注意compose的使用：之前没了解过是个串联变换的管道，进行连续的变化，总结一些常见预处理变化以及顺序的重要性

```
from torchvision import transforms

# 训练时的变换管道
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.08, 1.0)),  # 1. 随机裁剪缩放
    transforms.RandomHorizontalFlip(p=0.5),                # 2. 随机水平翻转
    transforms.ColorJitter(brightness=0.2, contrast=0.2), # 3. 颜色抖动
    transforms.ToTensor(),                                  # 4. 转张量
    transforms.Normalize(mean=[0.485, 0.456, 0.406],        # 5. 标准化
                         std=[0.229, 0.224, 0.225])
])

# 测试时的变换管道（确定性的）
test_transform = transforms.Compose([
    transforms.Resize(256),           # 1. 短边缩到256
    transforms.CenterCrop(224),        # 2. 中心裁剪224x224
    transforms.ToTensor(),             # 3. 转张量
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# 使用
img = Image.open('cat.jpg')
img_train = train_transform(img)  # 每次调用结果不同（随机增强）
```

```
img_test = test_transform(img)    # 每次调用结果相同
```

比如肯定要先totensor再normalize等等。

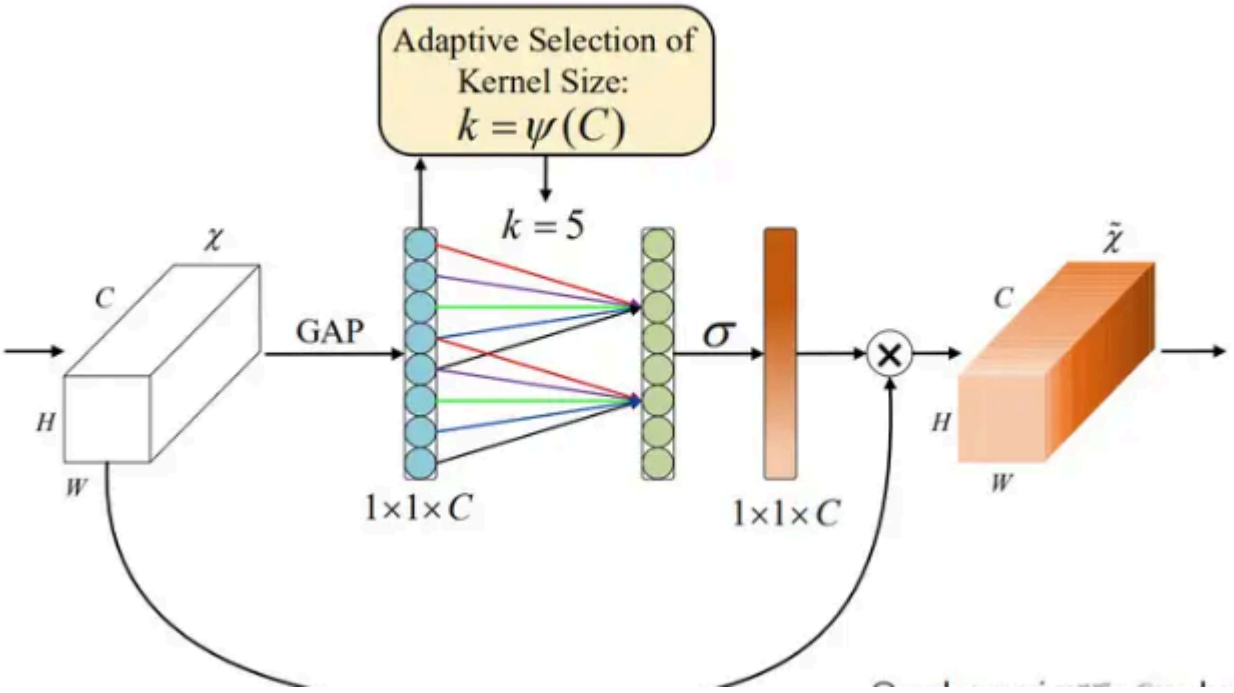
III.epoch和batch的区别

epoch是训练了所有的全部的数据之后就叫一个epoch

batch就是一次迭代参数所要训练的样本或者数据数量，然后可以理解为batch_size越小同样的数据量一个epoch要迭代参数，更新梯度的次数就越多，这有时候是好事情有时候也不是好事情，因为一次迭代的样本少了可能进入一些死胡同。所以batch_size的大小需要实际去动态调整对应的超参数。

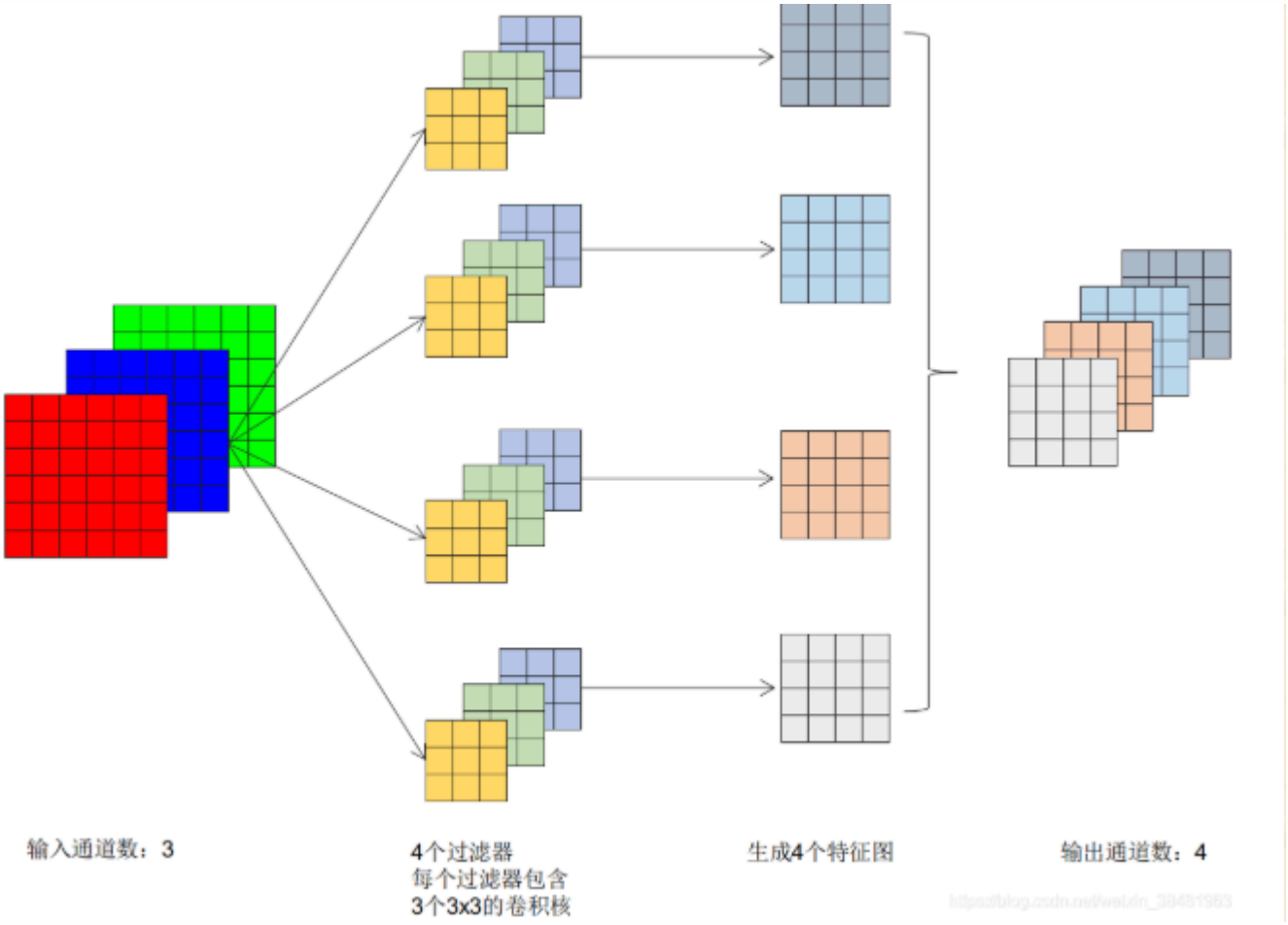
引申出来的iteration就是数据量÷batch*epoch就是跑完训练所需要的总迭代次数。

IV.1x1的卷积和 FC 有什么区别？主要起什么作用？



fc就是fully connected layer。就是所有的神经元对应相乘，两个矩阵相乘，一个权重矩阵一个参数矩阵，然后cnn卷积如果是1*1的。其实形容的就是kernel是1 * 1的一个矩阵，运算过程相当于fully connected

先讲一下卷积的通道数变化



一开始问的ai但是ai输出的话真的是模棱两可

$$192 \times 3 \times 3 \times 128 = 221184$$

不是把 192 个输入通道分配给 128 个输出通道，而是每个输出通道都读取全部 192 个输入通道，再将它们的卷积结果相加。

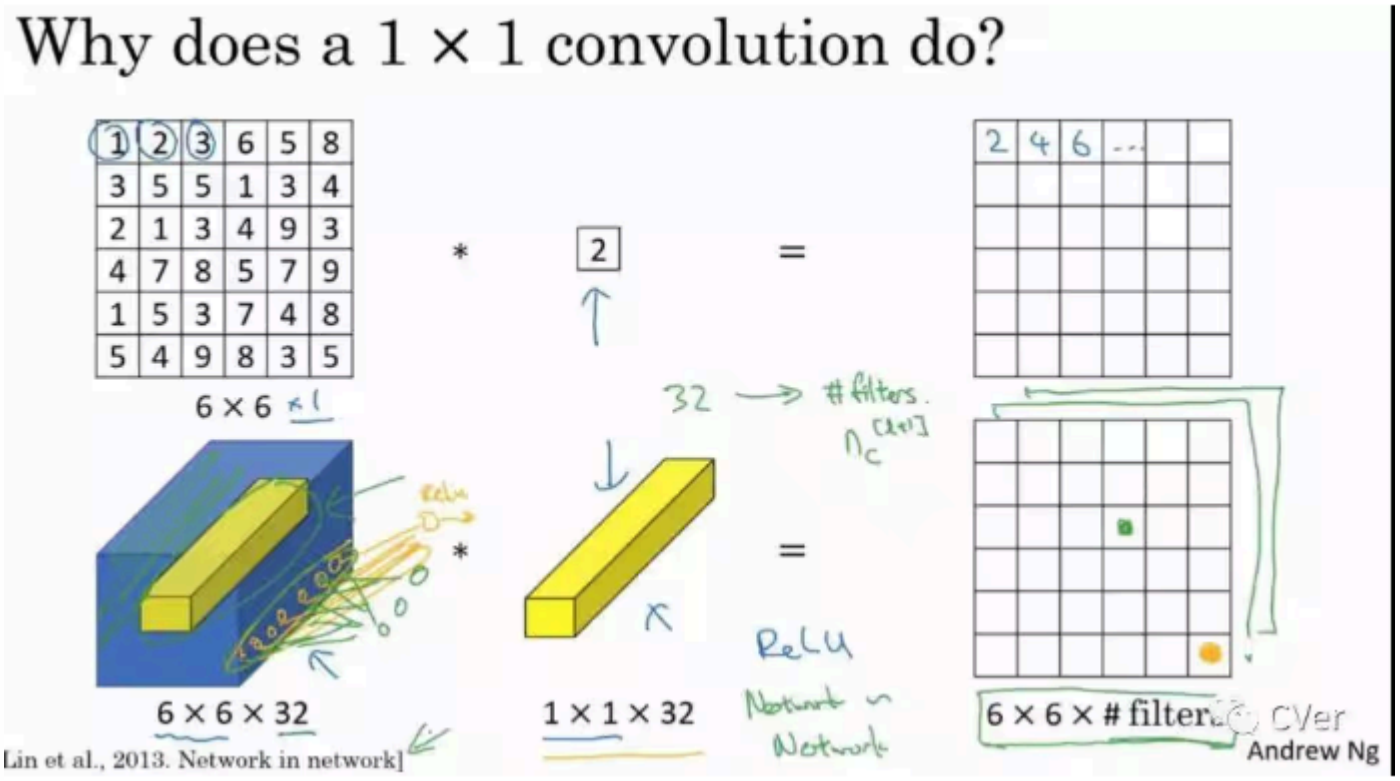
3. 如果确实想让每个输入通道单独计算呢?

这也是可以的，叫作**深度卷积（Depthwise Convolution）**。

例如：

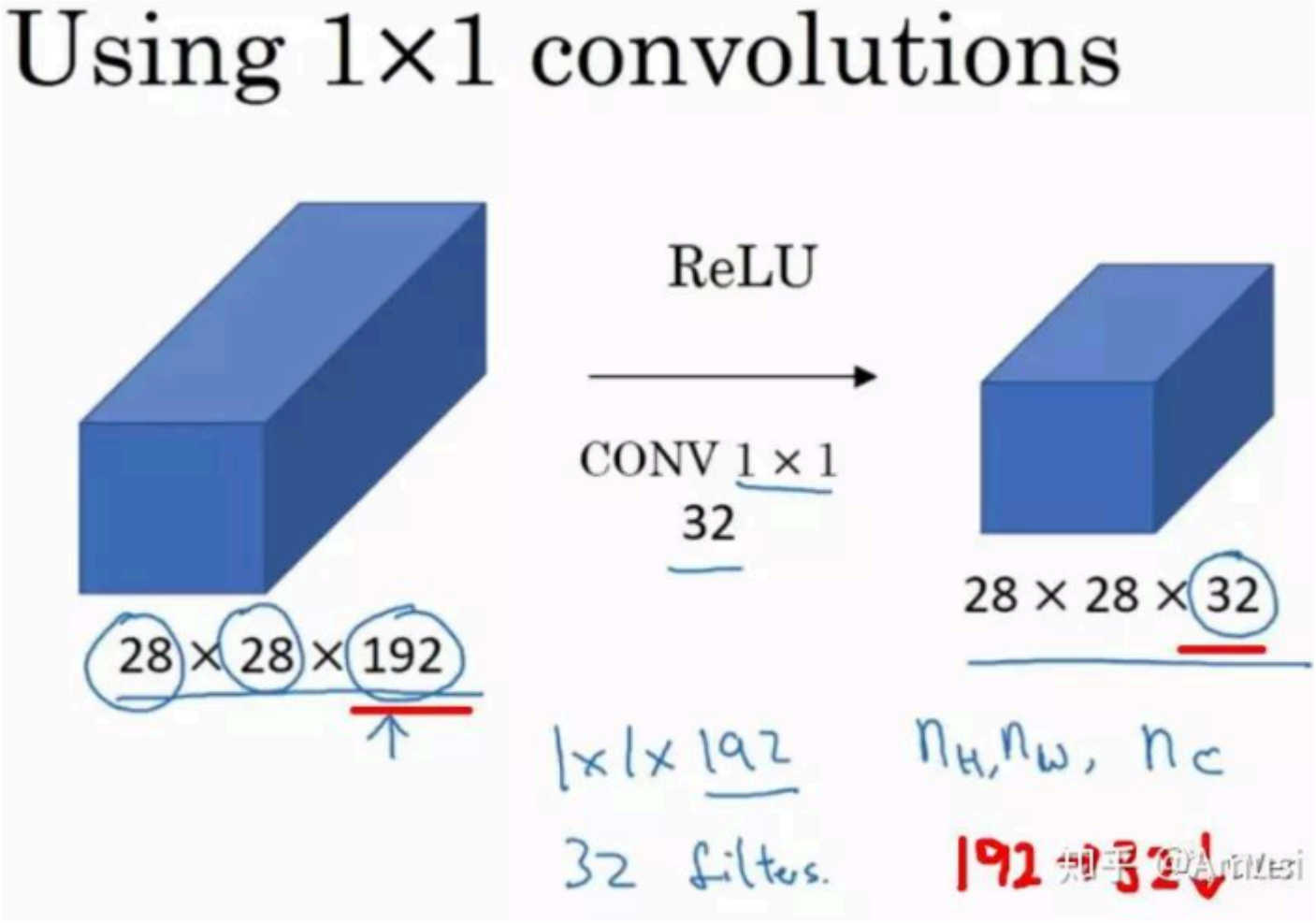
实际上kernel怎么做运算我们都知道矩阵对应位置相×相加但是为什么通道数量会在cnn进行的过程中发生变化？谁决定的，那么这里就可以从这个图片中看出来首先kernel的通道数必须和input的一致然后卷积之后相加就得到一个输出通道，然后用多个不同的kernel就会得到大量不同的输出channel然后就知道output cahnnel实际上等于kernel的数量，然后为什么要设计这么多的kernel和通道，实际上kernel是为了提取局部特征，设计不同的kernel的0 1 位就能把对应位置的特征提取出来，那通道的个数就是我想要提取不同特征的数量，实际上和机器学习中的特征工程有点类似。

知道这个之后我们就可以介绍为什么1*1的卷积核比fc好了



看这个图最终的输出结果是6 * 6 * filters。最终的输出通道数取决于我使用了几个单位kernel。那么这就可以用来降维升维获得不同的特征。这是他和fc的主要区别。

1.主要的作用是降维，当输入为6x6x32时，1x1卷积的形式是1x1x32，当只有一个1x1卷积核的时候，此时输出为6x6x1。此时便可以体会到1x1卷积的实质作用：降维。当1x1卷积核的个数小于输入channels数量时，即降维[3]。

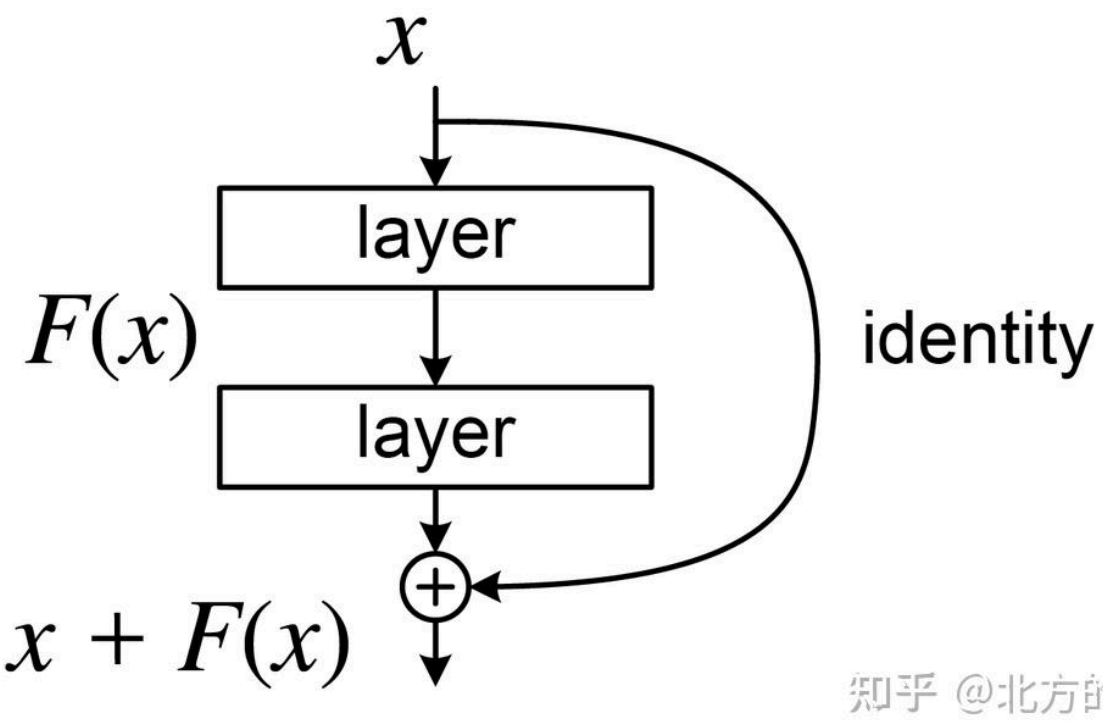


2.降维的同时我不同channel设计不同kernel带来的非线性的增加，一个filter对应卷积后得到一个feature map，不同的filter(不同的weight和bias)，卷积以后得到不同的feature map，提取不同的特征。

3.channel变换，后面接不同维度的kernel让线性组合传下去。

V.residual leanring 为什么能够提升准确率?

1.先考虑一下残差是什么。



残差就是这样把和目标输出的结果减去x作为参数去经过神经网络去更新。

2.残差的作用1：

之前提到过的梯度消失的问题残差就能很好的解决，反向传播的时候这个x就变成了+1，有效的抑制了梯度消失的问题，那么梯度消失如果减缓了，那可以训练更多的轮次和更精度的学习率，一定程度上可以提高准确率

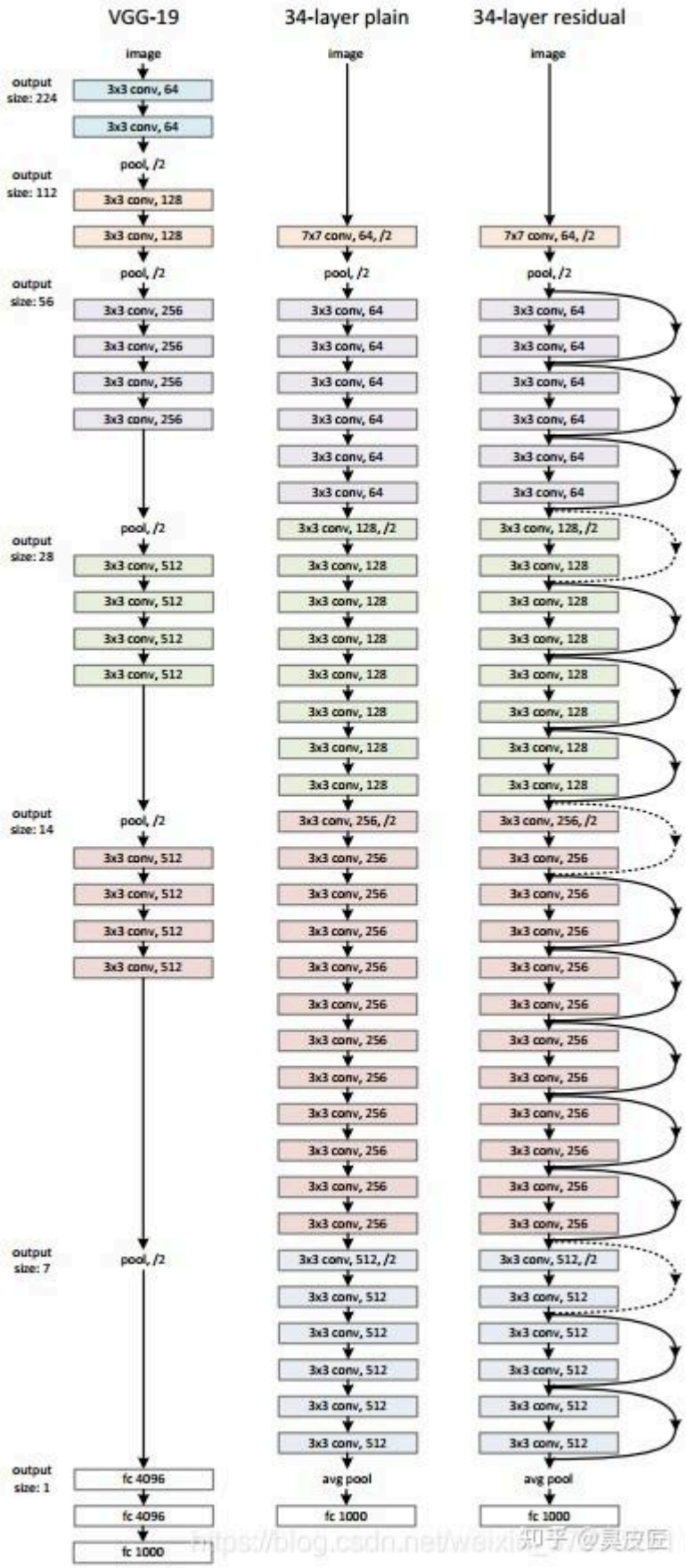
3.残差的作用2：

解决网络层数加深和过拟合带来的网络退化问题

简单地增加网络层数并不总能带来性能提升，反而可能导致性能下降。这种现象被称为“退化问题”

目标函数几乎是恒等映射，最终的目标就是让残差fx=0，趋近于0即可，这样预测值和实际值就理论最接近了。

其实就是让模型学习的难度降低了引入残差机制，这里放一个对比图，有无残差的模型结构


VGG-19
34-layer plain
34-layer residual
image
output size: 224
3x3 conv, 64
3x3 conv, 64
pool, /2
output size: 112
3x3 conv, 128
3x3 conv, 128
pool, /2
output size: 56
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
pool, /2
output size: 28
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
output size: 14
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
output size: 7
fc 4096
fc 4096
fc 1000
output size: 1
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 256, /2
3x3 conv, 256
3x3 conv, 512, /2
3x3 conv, 512
avg pool
fc 1000

VI.代码练习二里，网络和1989年 Lecun 提出的 LeNet 有什么区别?

1.首先看写的model的代码

```
class Net(nn.Module):
  def __init__(self):
    super(Net, self).__init__()
    self.conv1 = nn.Conv2d(3,6,5)
    self.pool = nn.MaxPool2d(2,2)
    self.conv2 = nn.Conv2d(6,16,5)
    self.fc1 = nn.Linear(16*5*5,120)
    self.fc2 = nn.Linear(120,84)
    self.fc3 = nn.Linear(84,10)
  def forward(self,x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = torch.flatten(x, 1)
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    return x

net = Net().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(),lr=0.001)
```

这个模型结构比较简单我就不划模型结构图了先看一下原始lenet结构

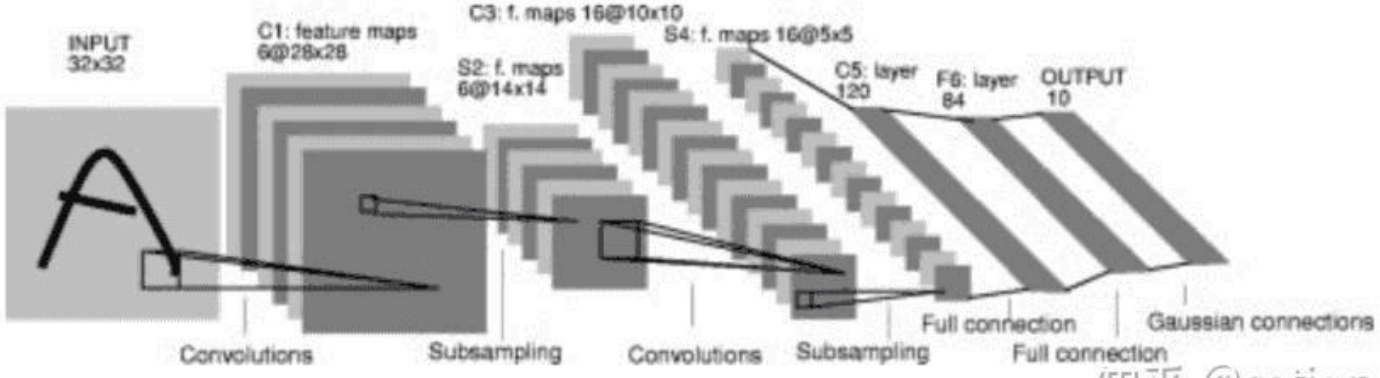


Fig. 2. Architecture of LeNet-5, a convolutional NN, here used for digits recognition. Each plane is a feature map, i.e., a set of units whose weights are constrained to be identical.

C1 → S2 → C3 → S4 → C5 → F6

2.不同

输入不同一个灰度图一个rgb

池化方式不同，原始是avgpooling，这里是maxpooling

激活函数lenet用的tanh，我们用的relu

linear层维度不一样

分类采用的函数不一样，我们是10 个 logits，配合交叉熵，他output是softmax

优化器用的也不一样，一个sgd一个adam

VII.卷积以后feature map 尺寸会变小，如何应用 Residual Learning?

1.首先理解feature map是什么，为什么feature map会发生变化以及变化成什么：

首先C ，H， W是feature map包括的主要内容。卷积的过程中可能导致H W变化缩放，也可能用不同的kernel让channel通道数发生变化。

2.这个map变化了那么残差上一层的就不能直接添加到后面，以为维度不一样

3.那怎么解决这个问题或者在这个变化的基础上怎么利用添加进去残差机制：

用1*1的kernel做卷积最简单直接，前文说到这个kernel不会改变HW参数，然后至于channel选取特定的kernel数量进行缩放即可。

但是这个是让feature map的变化最小，我们可以做填充把不同的维度补齐再用残差。

VIII.有什么方法可以进一步提升准确率?

1.换优化器比如adamw

2.加入weight-decay的同时加入余弦退火

3.warmup

4.数据增强进行更复杂的transform

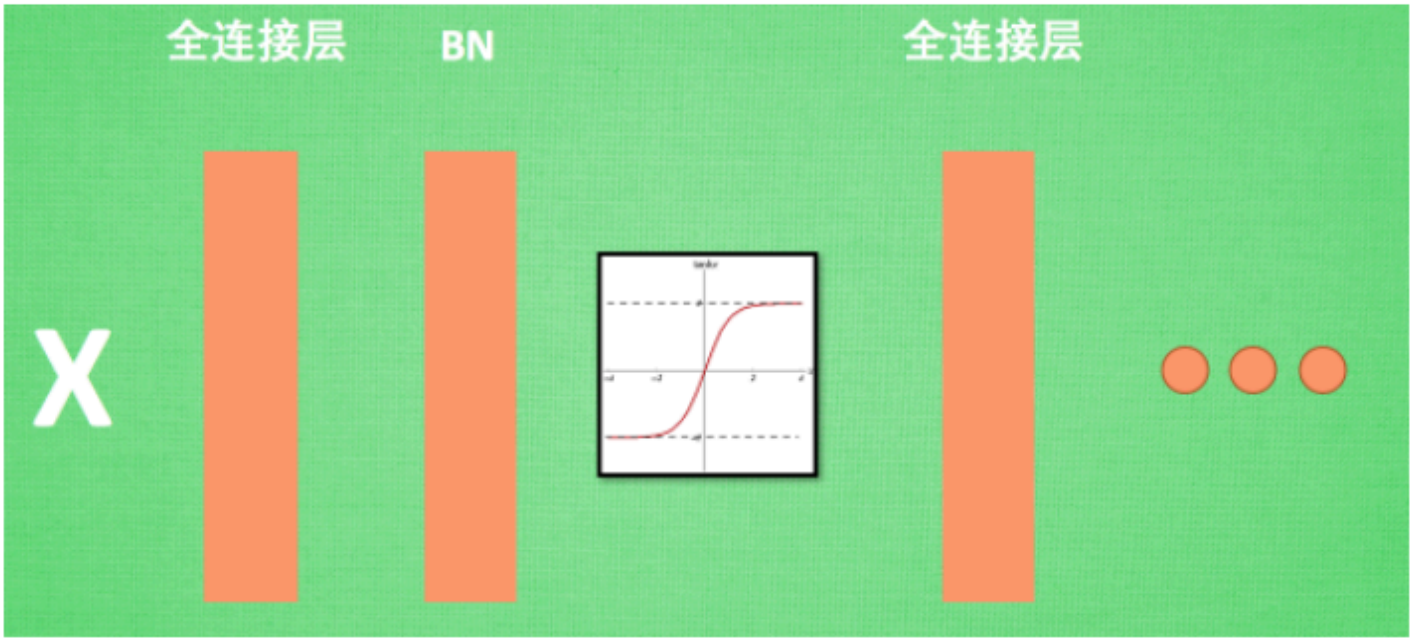
5.dropout（尤其是过拟合问题）

6.调整超参数---贝叶斯最优化啥的没用过不知道怎么样

7.Batch Normalization层

BN 添加位置

Batch normalization 的 batch 是批数据, 把数据分成小批小批进行 stochastic gradient descent. 而且在每批数据进行前向传递 forward propagation 的时候, 对每一层都进行 normalization 的处理,



8.优化模型结构加入一些注意力机制等等

9.优化训练算法label smoothing以及llrd等等算法都有不错的效果

IX.问题讨论

我感觉学的时候很逆天啊，比如我从自己学习开始到现在都对BN不是很了解太为什么这么大名鼎鼎，一直吹捧他的优异表现，但是一读发现讲解都是说他解决了什么Covariate shift问题协变量偏移，也看不懂。然后正则化的效果。还有就是测试集准确率一定比验证集低吗?